



// GEMACH FIELD MANUAL // AI ENGINEERING

HARNESS ENGINEERING

Zero to Hero. The complete learning path to designing the **scaffolding** around AI agents — context delivery, tool interfaces, planning artifacts, verification loops, memory, and sandboxes — the layer that decides whether an agent **succeeds or fails** on real work.

DISCIPLINE

Agent Harness Design

TRACK

5 Levels · 40+ Sources

EDITION

2026.1

// ORIENTATION

WHAT YOU'RE ABOUT TO LEARN

Harness engineering is the discipline of building everything *around* the model. The model can reason; the harness decides what it sees, what it can touch, how it plans, how it's checked, and what it remembers. The best harness engineers design these components knowing each one is a temporary crutch — every assumption you encode is something the model can't do *yet*.

// THE CORE PRINCIPLE

Every harness component exists because the model can't do something alone. Those assumptions expire as models improve. Design accordingly — favor scaffolding you can remove later over scaffolding you'll fight forever.

How this manual is structured

Five levels, each building on the last. Read the foundations, then work the primitives, then study real implementations, then harden and measure. Resources are tagged so you know what each one is:

READ

essays & docs — concepts and first-party guidance

REPO

code to study or adopt

BUILD

do-it-yourself milestone

PAPER

research — deeper / frontier findings

LV.1

FOUNDATIONS

LV.2

PRIMITIVES

LV.3

BUILD

LV.4

SCALE

LV.5

HARDEN

LV.6

SHIP

// CONTENTS

00	Orientation — what a harness is & the path	P.01
01	Level 1 · Foundations — the canonical essays	P.03
02	Level 2 · Design Primitives — the 12 components	P.04
03	Level 3 · Build Your First Harness	P.08
04	Level 4 · Scale & Orchestrate	P.09
05	Level 5 · Harden, Verify, Observe	P.10
06	Level 6 · Ship to Production — ops & FinOps	P.11
07	The Hero Checklist + Templates	P.12

// MENTAL MODEL

THE ANATOMY OF A HARNESS

Before the reading list, hold this map in your head. LangChain frames a harness as five primitives; the rest of this manual expands them into twelve working components. Everything you build slots into one of these.

Filesystem

Durable state and the surface where agent and human collaborate. Plans, diffs, artifacts live here.

Code Execution

Autonomous problem-solving without pre-designed solutions. The agent writes and runs code.

Sandbox

Isolation plus verification. Where untrusted action is contained and results are checked.

Memory

Cross-session persistence. What survives when the context window resets.

Context Management

Compaction against "context rot." The discipline of curating what the model sees as a finite, decaying resource — the single highest-leverage harness lever.

// CO-EVOLUTION WARNING

Models trained with a specific harness can overfit to its design. Your architecture choices have consequences beyond the immediate task — a harness is a long-term commitment, not a quick wrapper.

The loop everything runs on

At the center sits the agent loop: **observe** → **plan** → **act** → **verify**, repeat until done. ReAct (2022) defined the Thought / Action / Observation structure that virtually every harness still uses. When you read anything in this manual, ask: *which part of the loop does this improve?*

```
# one iteration of the agent loop
state = observe(filesystem, last_result)
plan = think(state, memory, context) # reasoning sandwich: think hard here
action = act(plan, tools) # tool call, bounded by permissions
result = verify(action, sandbox, tests) # the step most beginners skip
memory = compact(memory, result) # fight context rot every turn
```

// THE LESSON THAT REFRAMES EVERYTHING

LangChain moved a coding agent from rank 30 to top 5 on Terminal Bench 2.0 with **no model swap** — purely harness changes: structured verification, context injection, loop detection, and concentrating thinking at the plan and verify phases. Harness design is the primary performance lever, not model capability.

1 FOUNDATIONS

Read these before touching code. They define the vocabulary and the why.

[READ](#)

Harness Engineering

openai.com • martinfowler.com

Two essays, same name, different angles. OpenAI frames the discipline for an agent-first world. Martin Fowler's synthesis is the clearest conceptual map: three interlocking systems — **context engineering** (curating what the agent knows), **architectural constraints** (deterministic linters, structural tests), and **entropy management** (agents that repair doc drift). His "humans *on* the loop" framing — engineers who maintain the environment rather than inspect every output — is the job description.

Building Effective Agents

anthropic.com/research

The foundational architecture guide. When to use workflows vs. agents, and how to compose primitives. Start here for the design philosophy.

Harness Design for Long-Running Application Development

anthropic.com/engineering

The source of the core principle: every harness component assumes the model can't do something — and those assumptions expire. Read it for multi-session, sustained-work thinking.

Unrolling the Codex Agent Loop

openai.com

The canonical decomposition of one loop iteration — observe, plan, act, verify — exposing each harness component and where it can be improved.

The Anatomy of an Agent Harness

blog.langchain.com

The five-primitive breakdown from the mental model on the previous page. The structural reference you'll return to.

Run Long-Horizon Tasks with Codex

developers.openai.com

Introduces reusable planning artifacts — Plan.md, Implement.md, Documentation.md — as harness state. Your first concrete pattern.

Writing Effective Tools for Agents • Beyond Permission Prompts • Demystifying Evals

anthropic.com/engineering

The three "how to build the parts" essays — tool design is agent UX; structured authorization beats prompt-level trust; evals belong in the loop, not just after it. You'll revisit each at its own level.

What is an AI Agent? • A Practical Guide to Building AI Agents

ibm.com • openai.com

IBM's definitional piece anchors your design decisions to a clear model of what an agent is. OpenAI's April 2026 practical guide distills production patterns: single vs. multi-agent orchestration, tool design, and layered guardrails.

What makes a harness a harness • Architectural Design Decisions

[arxiv 2606.10106](https://arxiv.org/abs/2606.10106) • [2604.18071](https://arxiv.org/abs/2604.18071) • PAPER

The constitutive definition: a harness is a runtime layer with four necessary elements — agent loop, tool interface, context management, control mechanisms. Paired with an empirical study of 70 public systems across five design dimensions, this turns harness selection from a popularity contest into reasoned trade-offs.

2 DESIGN PRIMITIVES

Twelve components, organized by the problem they solve. This is the core curriculum — study them in order.

① Agent Loop

The control flow at the center of everything.

ReAct: Synergizing Reasoning and Acting

[arxiv 2210.03629](#) · PAPER

Required reading. Defines the Thought/Action/Observation loop. Understand why the loop is shaped the way it is.

LangGraph — Low Level Concepts

[langchain-ai.github.io](#) · READ

Models the loop as a directed graph: typed state, conditional edges, checkpointing. The most concrete treatment of loop control flow — termination conditions, branching on results, resuming mid-loop.

Improving Deep Agents with Harness Engineering

[blog.langchain.com](#) · READ

The rank-30-to-top-5 case study. The "reasoning sandwich" — maximum thinking at planning and verification — is a pattern you'll reuse constantly.

How Middleware Lets You Customize Your Agent Harness

[blog.langchain.com](#) · READ

Six composable hooks (`before_model`, `wrap_tool_call`, `after_agent...`) that intercept every loop stage. The reference design for cross-cutting concerns — PII redaction, retries, HITL — without touching core logic.

Extended Thinking

[docs.anthropic.com](#) · READ

Harness-critical: `budget_tokens` controls reasoning depth; thinking blocks **must** be preserved when passing tool results back, or multi-step reasoning silently breaks.

② Planning & Task Decomposition

Plan-and-Execute Agents

[blog.langchain.com](#) · READ

The canonical split: a planner generates the step list once; an executor works through it, replanning only when needed. The pattern most decomposition harnesses follow.

Effective Harnesses for Long-Running Agents

[anthropic.com/engineering](#) · READ

An initializer agent sets up the environment and hands off to a coding agent that makes incremental progress per session. Feature lists, git commits, and test gates become cross-session state — the reference for tasks bigger than one context window.

microsoft/TaskWeaver

[github](#) · REPO

Most complete plan-then-execute reference implementation, with stateful task tracking and a plugin system for injecting domain knowledge into the planner.

Choosing the Right Multi-Agent Architecture

[blog.langchain.com](#) · READ

Decision framework for subagents vs. skills vs. handoffs vs. router, with data: subagents process 67% fewer tokens than skills in multi-domain work because context isolation prevents bloat.

③ Context Delivery & Compaction

The highest-leverage primitive. Treat context as a finite, decaying resource.

Effective Context Engineering for AI Agents

anthropic.com/engineering · [READ](#)

Manage the full context state — system prompt, tools, MCP, history — as a curated resource. Reframes design as "what configuration of context produces the desired behavior?"

Compaction · Prompt Caching

platform.claude.com · [READ](#)

Server-side compaction cut tokens **84%** in a 100-turn eval. Prompt caching is the cheapest cost lever you have — cache system prompts, tool defs, long docs across turns.

Autonomous Context Compression

blog.langchain.com · [READ](#)

Shifts compression from fixed-threshold (harness-decides) to agent-decides — the agent compacts between tasks, not mid-subtask. Eliminates corruption of in-flight reasoning.

LLMLingua

[github/microsoft](https://github.com/microsoft) · [REPO](#)

Prompt compression toolkit, up to 20× with minimal loss; LLMLingua-2 adds speed for latency-sensitive loops. Drop it into the context-delivery layer.

A-RAG: Hierarchical Retrieval Interfaces

[arxiv 2602.03442](https://arxiv.org/abs/2602.03442) · [PAPER](#)

Reframes RAG as tool design: expose keyword/semantic/chunk-read tools and let the agent pull incrementally, instead of injecting documents upfront. Retrieval becomes a loop action.

④ Tool Design

Tool design *is* agent UX. Bad schemas cause more failures than weak models.

Writing Effective Tools for Agents

anthropic.com/engineering · [READ](#)

Naming, schemas, error messages, return conventions. The bedrock.

Tool Use (Claude) · Function Calling (OpenAI)

[platform docs](https://platform.openai.com/docs) · [READ](#)

Client vs. server execution is a foundational architecture decision. OpenAI's doc is the de-facto JSON-Schema standard — read before designing a cross-model interface.

Tool Annotations as Risk Vocabulary

blog.modelcontextprotocol.io · [READ](#)

The four hints (readOnly / destructive / idempotent / openWorld) feed permission decisions. Introduces the "**lethal trifecta**": private data + untrusted content + external comms — risk emerges from tool *combinations*, not single tools.

outlines · instructor

[github](https://github.com) · [REPO](#)

Structured-output reliability: outlines constrains decoding via regex/CFG/JSON-Schema; instructor maps Pydantic models with retry/validation feedback. Together they kill an entire class of parsing bugs.

⑤ Skills & MCP

Model Context Protocol + official servers + Inspector

modelcontextprotocol.io · [github](https://github.com) · [REPO](#)

The open standard for connecting agents to tools/data. Study the reference servers (GitHub, Slack,

⑥ Permissions & Authorization

Give agents the right access without relying on prompt-level trust.

Beyond Permission Prompts

anthropic.com/engineering · READ

Structured authorization patterns instead of natural-language permission text.

OWASP LLM06:2025 — Excessive Agency

genai.owasp.org · READ

The authoritative risk definition: over-provisioned functions, unnecessary permissions, missing approval gates. Your audit checklist for least privilege.

Claude Agent SDK — Configure Permissions

platform.claude.com · READ

Five-layer evaluation order (hooks → deny → mode → allow → canUseTool) and dontAsk deny-by-default for headless agents. Read the subagent bypassPermissions warning before any multi-agent deploy.

Claude Code Auto Mode: A Safer Way to Skip Permissions

anthropic.com/engineering · READ

Users approve 93% of prompts — making approvals meaningless. A two-stage classifier (fast gate, then reasoning only on flagged actions) plus deny-and-continue recovery is the safe-by-default reference.

Two Different Types of Agent Authorization

blog.langchain.com · READ

On-behalf-of (end-user credentials, identity mapping, per-user memory isolation) vs. fixed-credential (agent owns its account, needs HITL on high-risk actions). Different threat surfaces, different enforcement points.

⑦ Memory & State

What survives the context reset. Move durable facts out of the window.

Letta (MemGPT)

github.com/letta-ai · REPO

The reference stateful-agent architecture: three-tier memory (core / archival / recall) maps directly onto harness state design. Read their agent-loop redesign post.

mem0 · Zep

github.com · REPO

Drop-in production memory layers. mem0 = lowest integration cost for cross-session retention; Zep adds auto-summarization, entity extraction, semantic session search.

How We Built Agent Builder's Memory System

blog.langchain.com · READ

COALA three-tier memory on Postgres, exposed to the agent as a virtual filesystem. HITL approval gates every memory write — blocking prompt-injection via malformed writes.

Facts as First-Class Objects

arxiv.org/abs/2603.17781 · PAPER

Quantifies in-context memory failure: overflow at ~8,000 facts, 60% fact destruction during compaction, 54% drift across cascaded summaries. Knowledge Objects hit 100% accuracy at 252× lower cost. The case for structured retrieval over prompt engineering.

// PATTERN TO INTERNALIZE

GEMACH FIELD MANUAL

LEVEL 2 · PRIMITIVES (3/4)

If a fact must persist, it does not belong in the context window — it belongs in a structured store the agent retrieves on demand. Context is for what's needed *now*; memory is for what's needed *again*.

⑧ Task Runners & Orchestration

LangGraph + LangGraph 2.0

github/langchain-ai · REPO

The most widely adopted orchestration layer. Graph state machine, supervisor/subagent topologies, checkpoint persistence for resumable multi-day tasks. 2.0 ships Router/Supervisor/Subagent primitives so you stop hand-rolling coordination.

OpenAI Agents SDK · Google ADK · AutoGen · CrewAI

github · REPO

The framework field. Agents SDK = lightweight handoffs/guardrails. ADK = code-first with Runner/AgentTool. AutoGen = most comprehensive multi-agent reference. CrewAI = mixing autonomous (Crew) and scripted (Flow) execution.

PydanticAI

github/pydantic · REPO

Type-safe agents — tool defs and returns are Pydantic models, so structure mismatch becomes a type error, not a runtime bug. RunContext is the reference for session-scoped DI without globals.

LiteLLM

github/BerriAI · REPO

Unified proxy to 100+ providers behind one OpenAI-compatible interface — retry/fallback on 429/500, per-project cost & rate tracking. Swap models without touching orchestration code.

Building a C Compiler with a Team of Parallel Claudes

anthropic.com/engineering · READ

16 Claudes on one git repo, no central orchestrator — agents claim tasks via files, git forces collision resolution, a restart loop spawns fresh sessions. Key lesson: verbose test output pollutes context — emit summary lines, log detail to file.

⑨ Verification & CI · ⑩ Observability · ⑪ Debugging · ⑫ Human-in-the-Loop

The reliability quartet. You'll go deep on these at Level 5 — for now, know they exist and what each owns.

⑨ Verification & CI

promptfoo (YAML LLM testing, CI-native), **AgentBench** (multi-env eval), and OpenAI's skill-eval framework. Verification belongs **in the loop**, not just after.

⑩ Observability

OpenLLMetry, **Langfuse**, **Arize Phoenix**, **W&B Weave**. Trace every inference and tool call. Use OTel GenAI semantic conventions for portability.

⑪ Debugging / DX

AgentRx (Microsoft): trajectory normalization + constraint synthesis for root-cause analysis — **23.6%** better failure localization. Surfaces *why*, not just *that*.

⑫ Human-in-the-Loop

LangGraph interrupt/breakpoint/approve; Claude SDK canUseTool with "approve-with-changes." Fowler: humans *on* the loop is the only posture that scales.

// LEVEL 2 EXIT CRITERIA

You can name all twelve primitives, say which loop stage each touches, and name one tool or paper for each. You've installed and run at least one framework (LangGraph or Agents SDK) end-to-end.

3 BUILD YOUR FIRST HARNESS

Stop reading, start shipping. Learn by deconstructing a real harness, then assemble your own.

Study these implementations first REPO

Learn Harness Engineering (course)

walkinglabs.github.io · READ

A project-based curriculum on designing the environments, state, verification, and control systems behind Codex and Claude Code. The most approachable public course — each module builds a working artifact rather than summarizing concepts. Start here if you want a guided path.

rasbt/mini-coding-agent

github

A pure-Python coding agent (standard library only) implementing all six core harness components — live repo context, structured tools with permissions, context reduction, transcript resumption, bounded subagents — in one readable file. The clearest "how does a loop actually work" reference.

shareAI-lab/learn-claude-code

github

Step-by-step deconstruction of Claude Code as a harness (s01–s12). The best resource for seeing how loop, tools, skills, compaction, and task management compose in practice.

huggingface/smolagents

github

~1,000 lines of core — the entire harness readable in an afternoon. Study the code-agent pattern: the model writes Python that calls tools, eliminating JSON round-trips.

anthropics/claude-cookbooks · mastra-ai/workshop-mastracode

github

Official Anthropic notebooks for orchestrator-worker, parallel tool calling, compaction. Mastra's 11-topic workshop dissects every layer of a production coding agent: resumable harness, sandboxing, memory compaction, HITL, cost tracking.

Then build — the highest-leverage starting point

Claude Agent SDK

platform.claude.com · REPO

Exposes Claude Code's entire harness as a programmable API — built-in tool execution loop, PreToolUse/PostToolUse hooks, subagent definitions, allowedTools, session resumption. You **inherit the tool execution layer** instead of implementing it. Start here unless you have a reason not to.

// BUILD MILESTONE — "HELLO, HARNESS"

Ship a single-agent coding harness that: (1) runs the observe→plan→act→verify loop, (2) has 3–5 well-named tools with strict schemas, (3) writes a Plan.md and updates it each turn, (4) gates destructive tools behind a permission check, (5) runs a test after every change and feeds only summary lines back. That's a real harness. Everything after is refinement.

4 SCALE & ORCHESTRATE

One agent works. Now make many agents work together, survive long horizons, and cross system boundaries.

Multi-agent topology — choose deliberately

Choosing the Right Multi-Agent Architecture

[blog.langchain.com](#) · READ

Five-dimension matching table (distributed dev, parallelization, multi-hop, user interaction, latency). The most actionable guide for deciding when a **topology change — not a model change** — is the right lever.

Plan-and-Act · Agyn (team-based SWE)

[arxiv](#) · PAPER

Specialize planner and executor independently — different model sizes, tool access, reasoning budgets per layer. Plan-and-Act: 57.6% WebArena-Lite, 81.4% WebVoyager. Role-specific agents beat single-agent approaches.

Long-horizon survival

Effective Harnesses for Long-Running Agents

[anthropic.com/engineering](#) · READ

Initializer → coding-agent handoff with feature lists, commits, and test gates as cross-session state. The answer to "task exceeds one context window."

Ranking Engineer Agent (REA)

[engineering.fb.com](#) · READ

Meta's production harness for multi-day ML pipelines with **hibernate-and-wake checkpointing** — resume interrupted 6-hour tasks without losing context. Coherence across days when single turns exceed context limits.

Orchestrating Ambient Agents with Temporal

[temporal.io](#) · READ

Distributed-systems durability (retry semantics, activity monitoring) for agent workflows that outlast any single HTTP request or session.

Crossing boundaries — the protocol map

Developer's Guide to AI Agent Protocols

[developers.googleblog.com](#) · READ

Which protocol when an agent crosses a system boundary: **MCP** (tools/data), **A2A** (inter-agent via Agent Card discovery), **AG-UI** (agent→frontend streaming), plus UCP/AP2/A2UI. The practical decision map.

A2A Protocol · AG-UI · Composio

[github](#) · REPO

A2A = cross-framework agent interop. AG-UI = the missing real-time agent-to-UI layer. Composio wraps 250+ SaaS APIs as agent-ready actions with managed OAuth — one-line integration.

OmniRoute · The 2026 MCP Roadmap

[github](#) · [blog.mcp.io](#) · READ

OmniRoute: smart model routing for 40–60% cost reduction (cheap models for simple tasks). The MCP roadmap's transport & discovery changes affect how clients locate servers — read before investing in MCP infra.

5 HARDEN, VERIFY, OBSERVE

The difference between a demo and production. This is where most harnesses are won or lost.

Verification & CI gates

promptfoo · AgentBench

github · REPO

promptfoo: YAML-driven LLM testing with LLM-as-judge and native CI — add agent regression tests to a PR pipeline without a custom harness. AgentBench: study its environment isolation and task-definition format.

Testing Agent Skills Systematically with Evals

developers.openai.com · READ

Four dimensions (outcome, process, style, efficiency), JSONL trace capture for deterministic checks, then rubric grading only where needed. Add expensive LLM-as-judge checks only where they reduce real risk.

Agent Evaluation Readiness Checklist · Evaluating Skills

blog.langchain.com · READ

33-item lifecycle checklist. Key insight: separate **capability evals** (low pass rate, improvement target) from **regression evals** (near-100%, protection target). Curated skills took Claude Code from 9% → 82% task completion; ≤12 skills beat sprawl.

AgentAssay

arxiv 2603.02601 · PAPER

Binary pass/fail is useless for non-deterministic workflows. Behavioral fingerprinting caught 86% of regressions vs. 0% for binary; trace-first offline mode runs at zero extra inference cost.

Observability & debugging

Langfuse · OpenLLMetry · Arize Phoenix · W&B Weave

github · REPO

Trace every step. Langfuse = most adopted self-hostable (data residency). OpenLLMetry = OTel spans for the Grafana/Datadog/Jaeger ecosystem. Phoenix = self-hostable replay. Follow OTel GenAI conventions so traces stay portable.

AgentRx: Systematic Debugging for AI Agents

microsoft research · READ

Constraint-based diagnosis over manual log inspection — surfaces why an agent failed, where the violation originated, downstream consequences. 23.6% better localization.

Security & sandbox

How we contain Claude across products

anthropic.com/engineering · READ

Environmental isolation must be the primary boundary — model-layer defenses alone miss ~17% of overeager actions. The exfiltration-through-allowlist case study shows the weakest link is usually custom harness plumbing, not the sandbox.

OWASP LLM01/LLM06 · tldrsec/prompt-injection-defenses · Simon Willison

owasp · github · simonwillison.net · READ

Define the threat model with OWASP, select countermeasures from the tldrsec catalog. Willison's series explains why indirect prompt injection is uniquely dangerous: agents consume untrusted external content that can hijack actions. Watch the lethal trifecta.

E2B · Daytona · NVIDIA OpenShell · cua

github · REPO

Sandbox-as-primitive. E2B = Firecracker microVMs, ~150ms cold start. Daytona = long-lived stateful

6 SHIP TO PRODUCTION

72% of large companies run agents past the pilot stage; only 14% scale org-wide. The gap is operations infrastructure, not model choice.

The production reality

State of Agent Engineering 2026

langchain.com · READ

Survey of 1,300+ practitioners: 57% have agents in production, quality is the top barrier (32%), 89% have observability but only **52% run evals**. The clearest snapshot of where deployment maturity actually sits — and the eval gap is your opportunity.

Minions: Stripe's one-shot coding agents · Scaling Managed Agents

stripe.dev · anthropic.com · READ

Stripe ships 1,300+ PRs/week from unattended "minions": blueprints interleave deterministic nodes with agentic subtasks, one central 500-tool MCP server serves the fleet. Anthropic's managed-agents architecture decouples brain/hands/session — crash recovery via session replay cut p50 TTFT ~60%. Treat containers as cattle, not pets.

FinOps — the new unit economics

FinOps for Agents · Cost Optimization Guide 2026

infoworld.com · moltbook-ai.com · READ

Five budget guardrails enforced at the gateway: loop/step limits, tool-call caps, per-run token budgets, wall-clock timeouts, per-tenant budgets with anomaly alerts. Prompt caching gives ~90% discount on cached tokens; model routing saves 40–60%. New metric: **Cost-per-Accepted-Outcome**, not tokens consumed.

Reliability before launch

Backtesting AI Agents · How My Agents Self-Heal in Production

droid.io · blog.langchain.com · READ

Validate with **pass^k** (all 20+ trials must succeed), not **pass@k**. Dataset mix: 20% golden paths, 30% edge cases, 20% adversarial, 30% regression from real incidents. Then close the loop: detect regression → attribute to last deploy → dispatch a coding agent to open the fix PR automatically.

Managed platforms · governance

aws · cloud.google · READ

If you'd rather not self-host: AWS Bedrock AgentCore (serverless runtime, session isolation, managed memory) and Google's Vertex AI Agent Engine. AWS Agent Registry adds a governed catalog so teams stop rebuilding the same scaffolding in silos.

// THE SCALING GAP, IN ONE LINE

Pilots fail to scale because teams optimize the agent and neglect the operations plane — monitoring, evals, incident response, cost controls. Appoint the ops function *before* you expand, not after.

// SHIP-READY

THE HERO CHECKLIST

Before you call a harness production-ready, every box should be checked. Grouped by primitive.

☐	LOOP	Explicit observe→plan→act→verify with hard termination conditions and loop-detection middleware.
☐	LOOP	"Reasoning sandwich" — extended thinking concentrated at plan and verify; thinking blocks preserved across tool results.
☐	PLANNING	Persistent planning artifact (Plan.md) updated each turn; replanning only when the executor stalls.
☐	CONTEXT	Compaction strategy in place (server-side or agent-controlled); prompt caching on system prompt + tool defs.
☐	CONTEXT	Durable facts live in a structured store, not the window. No reliance on >~8k facts in context.
☐	TOOLS	Every tool: clear name, strict schema, useful error surface, risk annotations. Tool count minimized.
☐	TOOLS	Structured-output enforcement (outlines/instructor or native) — no ad-hoc JSON parsing.
☐	PERMS	Least-privilege audit vs. OWASP LLM06. Destructive tools gated; deny-by-default for headless.
☐	PERMS	Lethal-trifecta check: no single agent has private data + untrusted input + external comms unguarded.
☐	MEMORY	Cross-session state defined (commits, feature lists, test gates). Memory writes gated against injection.
☐	ORCHESTR.	Topology justified by isolation/parallelism need. Checkpoint-resume for long tasks. Provider fallback (LiteLLM).
☐	VERIFY	Verification runs in-loop. Capability evals separated from regression evals. CI gate blocks bad deploys.
☐	OBSERVE	Every inference + tool call traced (OTel GenAI conventions). Failure root-cause, not just failure detection.
☐	HITL	Human gates on high-risk actions with approve-with-changes; humans on the loop, not reviewing every output.
☐	FEEDBACK	Test/tool output emits summary lines only; full detail logged to file to protect agent context.

// DON'T START FROM SCRATCH — AND THE QUESTION THAT CATCHES MOST BUGS

The repo ships four reusable artifacts to copy: `AGENTS.md`, `PLAN.md`, `IMPLEMENT.md`, `HARNESS_CHECKLIST.md`. And for every component you built, ask: *what is this assuming the model can't do?* If the answer is "nothing anymore," delete it. The best harness is the smallest one that still works.

// FAST TRACK

30-DAY RUN SHEET

The fastest honest path from zero to a shipped harness. ~2 focused hours/day.

Week 1 — Foundations & the loop

- // **Days 1–2:** Read both *Harness Engineering* essays + Fowler. Write your own one-paragraph definition.
- // **Days 3–4:** ReAct paper + Unrolling the Codex Loop + Anatomy of an Agent Harness. Draw the loop and the five primitives from memory.
- // **Days 5–7:** Read Building Effective Agents + Effective Context Engineering. Install LangGraph or the Claude Agent SDK; run the quickstart.

Week 2 — Primitives in depth

- // **Days 8–9:** Tool Design — Writing Effective Tools + Tool Annotations. Refactor your tools to strict schemas.
- // **Days 10–11:** Context — Compaction + Prompt Caching + Autonomous Compression. Add caching and a compaction trigger.
- // **Days 12–14:** Permissions + Memory. Gate destructive tools; wire mem0 or Letta for cross-session state.

Week 3 — Build & deconstruct

- // **Days 15–17:** Work through shareAI-lab/learn-claude-code (s01–s12). Read smolagents core end to end.
- // **Days 18–21:** Ship the "Hello, Harness" milestone — full loop, Plan.md, permission gate, in-loop test with summary-only feedback.

Week 4 — Scale, harden & ship

- // **Days 22–24:** Multi-agent — Choosing the Right Architecture + a second specialized agent only if it earns its place. Add a sandbox (E2B or Daytona).
- // **Days 25–27:** Add promptfoo CI evals (capability vs. regression split) + Langfuse tracing + gateway budget caps.
- // **Days 28–30:** Run the Hero Checklist, backtest with pass^k, delete every crutch the model no longer needs. Ship.

// PREFER A GUIDED PATH?

Run the project-based **Learn Harness Engineering** course (walkinglabs) alongside this sheet — every module leaves you a working artifact.

// RESOURCE INDEX

THE FULL LIBRARY

The source list is large and updated weekly. This index maps every section to where it lives in your path, so you can go deep on any component on demand.

Foundations

OpenAI, Anthropic, Fowler & Böckeler, LangChain Anatomy, Red Hat, deepset. Plus the constitutive papers (what makes a harness a harness; design decisions across 70 systems).

Agent Loop

ReAct, LangGraph low-level, Codex App Server, Hooks, Extended Thinking, Middleware, scheduler-theoretic framework, statewright, deepclaude.

Planning

Plan-and-Execute, Plan-and-Act, TaskWeaver, LATS, Agyn, AdaptOrch, TDP, multi-agent architecture selection.

Context

Context Engineering, Compaction, Prompt Caching, LLMingua, A-RAG, Context7, semble, Token Savior, codebase-memory-mcp, headroom.

Tool Design

Writing Effective Tools, Tool Use/Function Calling, Tool Annotations, outlines, instructor, AutoHarness, MCP deployment patterns, CLI-Anything.

Skills & MCP

MCP + servers + Inspector, Streamable HTTP, 2026 roadmap, Code Execution with MCP, Composio, Microsoft Skills, superpowers, agentgateway, AIP.

Permissions

Beyond Permission Prompts, OWASP LLM06, Auto Mode, Agent SDK permissions, two auth types, OAP, IETF agent-auth, Nango, nah.

Memory

Letta, mem0, Zep, cognee, Stash, Knowledge Objects, MemArchitect, MAGMA, Codified Context, GitHub agentic memory.

Orchestration

LangGraph 2.0, OpenAI Agents SDK, ADK, AutoGen, CrewAI, PydanticAI, LiteLLM, Vercel AI SDK, Mastra, Temporal, Microsoft Agent Framework, Hive.

Verification & CI

promptfoo, AgentBench, OpenAI skill evals, Readiness Checklist, AgentAssay, DeepEval, Inspect AI, eval-driven development, sentrux.

Observability

Langfuse, OpenLLMetry, Arize Phoenix, W&B Weave, Logfire, Helicone, Braintrust, OTel GenAI conventions, BigQuery Agent Analytics.

Debugging

AgentRx, AgentOps, claude-devtools, AgentTrace, TraceCoder, AgentDebug, AgentPrism, LangSmith deep-agent debugging.

HITL

AWS HITL patterns, LangGraph interrupts, Claude SDK approvals, HITL Protocol, HiL-Bench, measuring agent autonomy, human judgment in the loop.

Security & Sandbox

How we contain Claude, E2B, Daytona, NVIDIA OpenShell, K8s Agent Sandbox, tldrsec defenses, Willison series, RAMPART, AI Harness Scorecard.

Evals

SWE-bench, tau-bench, Claw-Eval, infrastructure noise, AgentLens, eval-awareness, AI-resistant evals, STATE-Bench, OccuBench.

Meta-Harness

Meta-Harness paper + Stanford impl, AutoAgent, metaharness, harness-evolver, auto-harness, HyperAgents, Omnigent, Live-SWE-agent.

// ADJACENT COLLECTIONS TO TRACK

Awesome Context Engineering, awesome-claude-code, awesome-mcp-servers, awesome-ai-agents, VoltAgent's 363+ paper collection, and the ICLR 2026 MemAgents workshop. The frontier moves weekly — bookmark the source repo and watch its commits.



github.com/ai-boost/awesome-harness-engineering · CC0